DressMe

Objectif 1 : reconnaissance intelligente des vêtements

Fiche technique détaillée et explication de l'architecture

Analyse fondée sur le notebook `dress_me_obj1.ipynb`

28 octobre 2026

Table des matières

1 Objectif et périmètre 2

2 Vue d'ensemble de l'architecture 2

3 Environnement et configuration 2

4 Étape 1 : acquisition et préparation de l'image 3

5 Étape 2 : définition des catégories et normalisation 3

6 Étape 3 : création de la base de connaissances 3

7 Étape 4 : FashionCLIP, embeddings et kNN 4

8 Étape 5 : fine-tuning supervisé optionnel 4

9 Étape 6 : Qwen2.5-VL et prompt engineering 4

10 Étape 7 : contrôles de fiabilité et score d'hallucination 5

11 Étape 8 : agent et politique de décision 5

12 Étape 9 : fusion des attributs 5

13 Étape 10 : perception multi-articles optionnelle 5

14 Étape 11 : démonstration et traçabilité 6

15 Étape 12 : protocole d'évaluation 6

16 Limites et améliorations à défendre 6

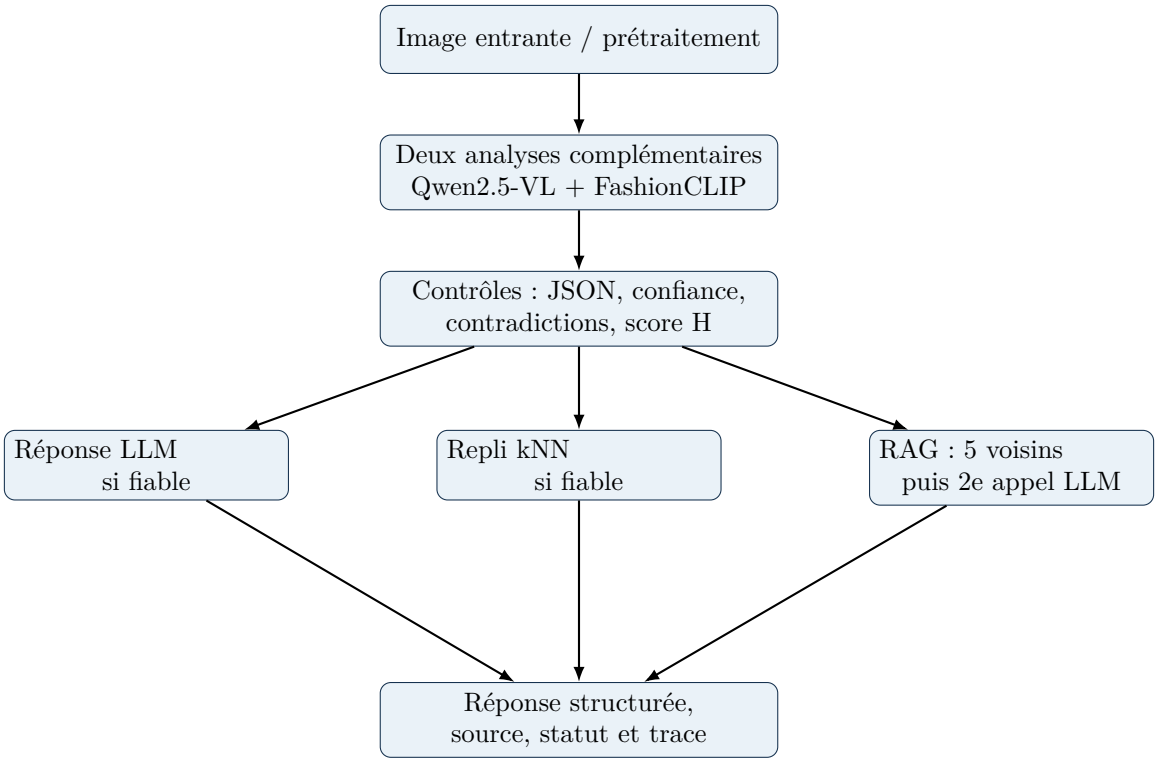
17 Synthèse technique 6

1 Objectif et périmètre

L'objectif est d'analyser une photographie de vêtement et de produire une catégorie principale et des attributs (couleur, genre, saison, occasion), avec une confiance déclarée, une justification visuelle et une trace des décisions. Le système met en oeuvre un agent qui choisit entre réponse directe du modèle multimodal, repli sur recherche par voisins et révision assistée par récupération documentaire (RAG). Le notebook décrit également une option de détection de plusieurs articles et une adaptation supervisée de l'encodeur visuel.

Distinction importante. Le notebook contient le code, des paramètres et des procédures d'évaluation. Il ne suffit pas à attester qu'une expérience a réussi ou qu'un score chiffré a été obtenu : ces affirmations nécessitent les sorties d'exécution ou les fichiers de résultats.

2 Vue d'ensemble de l'architecture



La recherche des voisins est exécutée au début de `run_agent`, même si le LLM répond directement : elle sert également à vérifier les contradictions. Le schéma présente les principales voies de décision, sans prétendre représenter chaque branche de secours.

3 Environnement et configuration

Le notebook utilise Python, PyTorch, Transformers, Accelerate, BitsAndBytes, NumPy, Pandas, Pillow, scikit-learn et Matplotlib, principalement dans Google Colab. Les fichiers sont enregistrés dans Google Drive si le montage réussit, sinon dans un répertoire local. Le dossier de sortie est `agent_final`. Le GPU CUDA est utilisé lorsqu'il est disponible.

Élément	Valeur configurée	Rôle
LLM	Qwen2.5-VL-7B-Instruct	Analyse multimodale et génération JSON ; chargement 4 bits configuré.

Encodeur	patrickjohncyh/fashion-clip	Extraction des vecteurs visuels pour recherche.
MAX_NEW_TOKENS	200	Limite de génération.
CONF_MIN	0,70	Seuil minimal de confiance déclarée du LLM.
KNN_K / RAG_K	10 / 5	Voisins pour vote et exemples pour le second prompt.
KNN_MIN	0,60	Seuil du vote pondéré kNN.
ATTR_MIN	0,70	Consensus requis pour contradiction d'attribut.
H_HIGH	0,50	Seuil de déclenchement du RAG.
DUP_SIM	0,97	Filtrage des quasi-doublons test/base.
KB_PER_CAT	300	Plafond configuré d'images de base par catégorie.
N_TEST_PER_CAT	8	Cible de sélection test par catégorie.
ANALYZE_OUTFITS	False	Détection multi-articles désactivée par défaut.

Ces valeurs sont des choix initiaux du code, non des hyperparamètres validés expérimentalement.

4 Étape 1 : acquisition et préparation de l'image

La fonction `load_image` accepte un chemin ou un objet Pillow. Elle convertit l'image en RGB, agrandit les petites images jusqu'à une dimension minimale configurée de 384 pixels et réduit les images dont le plus grand côté dépasse 1024 pixels. Le redimensionnement utilise LANCZOS. `pad_square` peut placer une image sur un fond blanc carré. `img_key` construit une empreinte MD5 sur une version réduite 64×64 pixels, utile à l'identification technique ; ce n'est pas une preuve d'identité visuelle absolue.

5 Étape 2 : définition des catégories et normalisation

Le code définit 17 catégories : `tshirt, shirt, top, sweater, jacket, blazer, coat, dress, skirt, jeans, pants, shorts, underwear, shoes, bag, accessory, traditional_garment`. Un dictionnaire `ART` convertit les types d'articles du catalogue en catégories cibles. Des tables normalisent aussi les couleurs et les attributs. Cette correspondance évite de confondre les libellés d'origine du dataset avec les classes effectivement prédites.

Point de vigilance. Les classes `coat` et `traditional_garment` ne sont pas représentées dans le catalogue historique selon les commentaires du notebook. Leur reconnaissance demande des exemples adaptés et une évaluation distincte.

6 Étape 3 : création de la base de connaissances

`build_catalogue` prépare les métadonnées du catalogue Fashion Product Images. Le programme construit une base d'images étiquetées et réserve des images pour le test. Les catégories, couleurs, genres, saisons et occasions proviennent des champs du catalogue et de leur normalisation, sans annotation manuelle de ce catalogue. Les embeddings sont calculés avec FashionCLIP et enregistrés avec les métadonnées.

Prévention de fuite. Les images du test sont exclues de la base de récupération ; un contrôle de similarité supprime aussi les images de base trop proches d'images test selon le seuil `DUP_SIM=0.97`. Il faut conserver cette séparation lors de toute reconstruction de l'index.

Fichiers. `kb_meta.csv` stocke les métadonnées, `kb_emb.npy` les vecteurs, et `test_meta.csv` les références du test. Le nombre réel d'images dépend du dataset disponible et des filtres : il ne se déduit pas simplement des plafonds configurés.

7 Étape 4 : FashionCLIP, embeddings et kNN

FashionCLIP transforme une image en représentation vectorielle permettant de comparer sa proximité avec les images de référence. La classe `FashionClipEmbedder` charge `CLIPModel` et `CLIPProcessor` ; la classe `Retriever` effectue la recherche. Pour une image requête q, on compare son embedding aux embeddings du catalogue. Le vote `knn_vote` additionne les poids `w` des voisins de chaque catégorie :

$$S(c) = \sum_{i \in \mathcal{N}_k \,:\, y_i = c} w_i, \qquad \hat{c} = \arg\max_c S(c).$$

Le champ `conf` du vote représente le poids de la catégorie gagnante selon l'implémentation. Ce score ne doit pas être assimilé sans validation à une probabilité statistiquement calibrée. Le vote est jugé suffisamment fort lorsqu'il atteint 0,60.

8 Étape 5 : fine-tuning supervisé optionnel

La section 3B du notebook prévoit d'adapter la dernière couche vision et la projection de FashionCLIP, ainsi qu'une tête de classification. Le jeu d'entraînement mélange des exemples génériques hors test et, s'ils existent réellement, des images tunisiennes documentées. Le fichier attendu est `tunisian_dataset/labels.csv`, avec `image_path` et `category` obligatoires ; `occasion`, `event`, `origin` et `group_id` sont facultatifs.

```
image_path,category,occasion,event,origin,group_id
images/example.jpg,traditional_garment,traditional,wedding,tunisia,shoot_01
```

Cette ligne illustre uniquement le format ; elle n'atteste pas l'existence d'une image annotée. Le fractionnement par groupes limite la présence de photos d'une même séance dans entraînement et validation. La configuration prévoit 5 époques, un batch de 8, un taux d'apprentissage vision de 10^{-5}, un taux tête de 2×10^{-4}, une patience de 2 et 20% pour la validation. Les exemples tunisiens sont répétés deux fois dans le train, pas dans la validation. Les poids et l'historique peuvent être sauvegardés, puis l'index de récupération réactualisé.

Interprétation. La présence du code de fine-tuning ne démontre pas qu'il a été exécuté ni qu'il améliore les résultats. Le notebook prévoit un test externe, avec accuracy, macro-F1 et rapport par classe, lorsque la tête entraînée est disponible.

9 Étape 6 : Qwen2.5-VL et prompt engineering

Le modèle multimodal reçoit l'image et un prompt de classification. Le message système impose une analyse fondée sur ce qui est visible, interdit d'inventer une marque ou une matière et autorise `unknown`. Le prompt v1 précise le rôle, la taxonomie, les règles de départage et le format JSON strict. Le modèle doit d'abord citer un indice visuel court, puis choisir sa catégorie et ses attributs.

```
{
  "evidence": "visible cues",
  "category": "dress",
  "alternative": "skirt",
  "color": "blue",
  "gender": "unknown",
  "season": "unknown",
  "occasion": "formal",
  "confidence": 0.72
}
```

Cet objet est un exemple explicatif et non une sortie mesurée. `parse_answer` et `extract_json` extraient et normalisent la réponse ; les synonymes de catégories et d'attributs sont harmonisés. Le guide de confiance du prompt distingue cas clairs (au-dessus de 0,85), plausibles (0,60–0,85) et incertains (en dessous de 0,60), mais l'auto-évaluation du LLM n'est pas une calibration vérifiée.

10 Étape 7 : contrôles de fiabilité et score d'hallucination

`accuracy_ok` exige un JSON valide et $confidence \geq 0.70$. Le contrôle `hallucination` calcule un score heuristique $H \in [0, 1]$:

$$H = \min\big(1,\ 0.25I_{vocab} + 0.60I_{knn} + 0.20I_{couleur} + 0.20I_{genre}\big).$$

Si la sortie est invalide, $H = 1$ immédiatement. I_{vocab} signale des valeurs hors vocabulaire ; I_{knn} signale un désaccord de catégorie avec un kNN suffisamment sûr ; les deux derniers indicateurs signalent une contradiction de couleur ou de genre avec le consensus des voisins au seuil 0,70. Le seuil $H \geq 0.50$ conduit à la révision RAG.

Limite méthodologique. Ce score mesure des contradictions détectables par les règles du programme. Il ne mesure pas toutes les hallucinations possibles et peut être influencé par des erreurs du catalogue ou du kNN.

11 Étape 8 : agent et politique de décision

La fonction `run_agent` orchestre le système : elle calcule les voisins et leur vote, appelle le LLM avec le prompt v1, vérifie la validité, la confiance et le score H, puis choisit une route.

1. Si le JSON est valide, la confiance atteint 0,70 et $H < 0.50$, la réponse LLM est retenue (source `llm`).
2. Si $H < 0.50$ mais la première réponse est insuffisamment fiable, le kNN est retenu lorsque son vote atteint 0,60 (source `knn_fallback`).
3. Si $H \geq 0.50$, ou si le kNN ne permet pas un repli fiable, le système construit un prompt v2 avec cinq voisins du catalogue, l'analyse précédente et les anomalies détectées. Qwen2.5-VL est appelé une seconde fois.
4. Après la révision, de nouveaux contrôles déterminent la réponse finale ou un statut de revue. Le code conserve les voies de secours et les traces pour analyser les décisions.

Le RAG ne constitue pas ici un second modèle génératif : il fournit au même LLM des exemples visuellement proches issus du catalogue. L'agent choisit quand utiliser cette information.

12 Étape 9 : fusion des attributs

La fonction `merge_attributes` ne traite pas tous les champs de la même manière. Pour `season` et `occasion`, les voisins du catalogue sont privilégiés si leur consensus atteint 0,50, car ces informations relèvent souvent de conventions. Pour `color` et `gender`, la sortie visuelle du LLM est conservée sauf si elle est inconnue ou contredite par un consensus voisin suffisamment fort (0,70). Cette fusion introduit une dépendance aux conventions du catalogue, qu'il faut documenter lors de l'interprétation.

13 Étape 10 : perception multi-articles optionnelle

Le notebook contient `perceive`, une étape facultative de localisation d'articles dans une tenue complète, associée à un détecteur OWL-ViT. Les groupes de détection couvrent haut, veste, bas, robe, chaussures et sac. Les boîtes sont filtrées selon des seuils de score, de surface et de recouvrement. L'objectif est de soumettre chaque article retenu à l'agent. Cette option est désactivée dans la

configuration fournie : `ANALYZE_OUTFITS=False`. Elle ne doit donc pas être présentée comme le mode principal testé sans preuve d'exécution.

14 Étape 11 : démonstration et traçabilité

La section A du notebook permet de téléverser une photo avec `files.upload()`, puis d'appeler `analyze_photo` et `show_result`. L'affichage prévoit la photographie, la décision, la source, la trace des outils et les voisins du catalogue. Les informations de traçabilité comprennent notamment la catégorie, le statut, la confiance, H, les alertes, le nombre d'appels LLM et la latence.

15 Étape 12 : protocole d'évaluation

La section B parcourt les images réservées au test et écrit progressivement les résultats dans un fichier JSONL, ce qui permet la reprise après interruption. Les mesures prévues sont : exactitude finale de la catégorie, exactitude de la première passe LLM, exactitude avec regroupement de classes proches, exactitude sur les cas répondus, taux de revue, nombre moyen d'appels LLM, latence moyenne, performances par route et confusions. Les intervalles de confiance à 95% pour les proportions utilisent la méthode de Wilson.

$$\text{Accuracy} = \frac{\#\{\hat{y}_i = y_i\}}{N}, \qquad \text{ReviewRate} = \frac{N_{review}}{N}.$$

Le notebook prévoit aussi le rapport de classification du modèle fine-tuné et l'inspection d'images mal classées. Les fichiers de sortie comprennent `agent_eval_v1.jsonl`, `agent_eval_v1.csv`, `summary_v1.csv` et `config_v1.json`, selon `RUN_TAG`. La section évoque 120 images de test, mais la taille effective doit être vérifiée dans `test_df` et les fichiers générés.

16 Limites et améliorations à défendre

1. Régler les seuils sur un ensemble de développement indépendant, plutôt que sur le test final.
2. Mesurer la calibration des scores et ne pas présenter `confidence` comme une probabilité garantie.
3. Comparer les résultats du LLM seul, de l'agent complet et des différentes routes sur le même test.
4. Tester les photos réelles (éclairage, arrière-plan, plusieurs articles), au-delà des photos de catalogue.
5. Vérifier séparément les catégories peu ou pas représentées, notamment les vêtements traditionnels tunisiens.
6. Contrôler la qualité des labels, le déséquilibre des classes et les éventuels quasi-doublons entre les ensembles.
7. Conserver des exemples d'échecs, les traces et les paramètres de chaque exécution pour rendre les conclusions reproductibles.

17 Synthèse technique

L'Objectif 1 met en place une architecture d'analyse d'images où Qwen2.5-VL produit une première interprétation, FashionCLIP fournit des voisins visuels, et un agent applique des contrôles explicites pour accepter, remplacer ou réviser la décision. La base de référence est construite à partir des annotations du catalogue ; une adaptation supervisée aux images tunisiennes est prévue lorsque des données réelles sont disponibles. Le système inclut une procédure d'évaluation et des mécanismes de traçabilité. Les conclusions quantitatives doivent être renseignées uniquement à partir des sorties réelles du notebook.